Event Types: Notifications, State Transfer, Commands

what a message should carry

Phase 4 videos on YouTube

In one line: an event notification says something happened and readers fetch details, event-carried state transfer includes the data so readers don't call back, and a command asks one service to do something; name events in the past tense, commands in the imperative, and choose payloads deliberately.

1 The callback storm

Your order service publishes an event called `OrderPlaced` containing only the order ID. The shipping team reads it and immediately calls the order service back to get the address. So do billing and analytics. Every order now creates three extra API calls, and when the order service is slow, every consumer is slow too: the coupling you tried to remove is back.

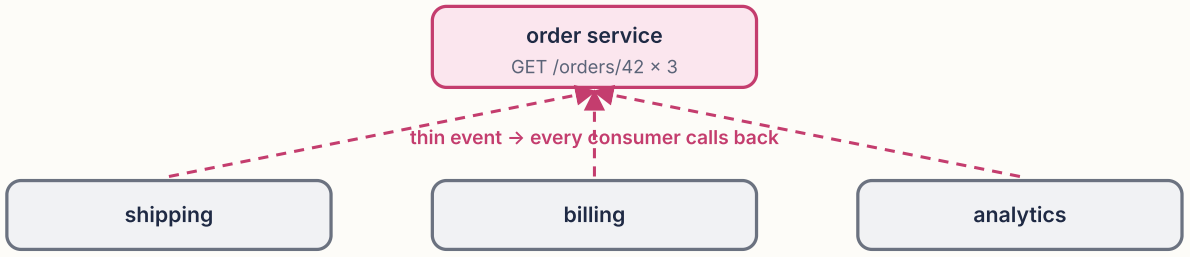


Figure 1. Thin events make consumers call the producer, re-coupling them.

2 Three kinds of message

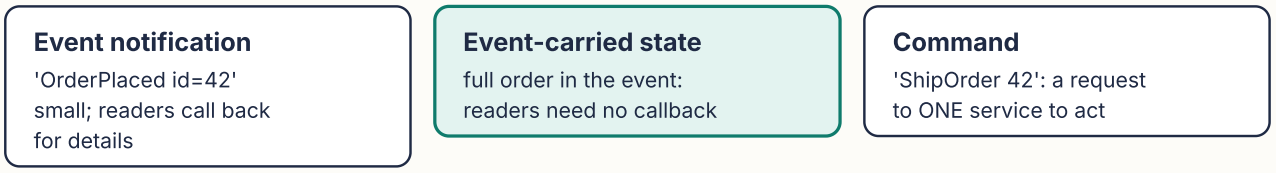


Figure 2. Events state facts in the past tense; commands request an action.

Event notification ('OrderPlaced, id 42'): tiny and stable, but consumers need callbacks for details, and the details may have changed by then. **Event-carried state transfer**: the event includes what consumers need (items, prices, address), so they can act, or keep their own copy, with no callback. Bigger messages and more schema to evolve (lesson 39), but real decoupling. **Commands** ('ShipOrder'): addressed to one owner, which may refuse; they're requests, not facts, and often go to a queue rather than a shared topic.

WORKED EXAMPLE: ORDERPLACED WITH THE FULL PAYLOAD

The order service publishes `OrderPlaced` with the order ID, customer ID, line items with prices, the total and the shipping address, plus an `event_id`, schema version and `occurred_at`. Shipping creates a shipment from the event alone; billing records revenue; analytics stores it. None of them call the order service, and they all see the order exactly as it was when placed, even if the customer edits their address later. Sensitive data (full card numbers) stays out of the event.

```
{ "type": "OrderPlaced", "event_id": "e-912", "version": 2,
  "occurred_at": "2026-12-05T09:01:12Z",
  "order": { "id": 42, "customer_id": 17, "total": 59.80,
    "items": [{"sku": "BK-1", "qty": 2, "price": 29.90}],
    "shipping_address": { "city": "Pune", "pin": "411001" } } }
```

Figure 3. An event-carried state transfer payload with metadata.

3 Naming and design

- Events: past-tense facts in business language: `OrderPlaced`, `PaymentFailed`, not `OrderTableUpdated`.
- Commands: imperative: `ReserveStock`, `SendEmail`.
- Always include an event ID (for dedup), a type and version, a timestamp, and a correlation ID.
- Prefer **domain events** designed for consumers over raw database change events, which leak your schema.

COMMON MISTAKES

- Thin events that force every consumer to call back.
- Commands disguised as events ('OrderShouldBeShipped').
- Publishing internal table changes as the public contract.
- Putting secrets or excessive personal data in widely-read events.

4 Key terms

Event: a fact that happened, past tense

Event notification: minimal event; details fetched separately

Domain event: business-level event, not a DB change

Command: a request for one service to act

Event-carried state transfer: event includes the needed data

Correlation ID: ties all messages of one flow together

5 Interview questions

Q1. What's the difference between an event and a command? BASIC

An event states a fact that already happened and can have many consumers; a command asks one specific service to do something and can be rejected.

Q2. What is event-carried state transfer? BASIC

Including the relevant data in the event so consumers can act or keep a local copy without calling the producer.

Q3. Consumers keep calling your service after every event. What do you change? SCENARIO

Add the fields they need to the event (event-carried state transfer), version the schema, and keep sensitive fields out.

Q4. Should you publish Debezium row changes directly to other teams? SCENARIO

Usually not as the public contract: they expose your table structure. Transform them into domain events, or use an outbox with designed payloads.

Q5. What's the downside of fat events? TRICKY

Larger messages, more fields to keep compatible, data copies spread across services, and a risk of leaking sensitive data.

Q6. Can a consumer trust that event data is current? TRICKY

It's current as of the event time. If it needs the latest state, it must apply later events in order or query the owner.

Next: SD-I-013 · Event Sourcing